

Detailed Code & File Architecture Documentation

A Deep-Dive into Every Component of the Multi-Document RAG System

Project: Multi-Document RAG Assistant

Framework: Flask, Pinecone Vector DB, SQLite, SentenceTransformers, Google GenAI SDK

Version: 2.0 (Verified & Stable)

Date: June 5, 2026

Compiler: Advanced Agentic Coding System

1. System Design and Layers

The RAG application is cleanly structured into multiple architectural layers: Configuration, Database, Services (Ingestion, Search & Generation), API Routing, and Presentation (UI). This breakdown ensures concerns are separated, allowing modules like the vector database (Pinecone) or the LLM model (Gemini) to be updated or swapped with minimal disruption to other components.

2. Application Entrypoint & Configuration

2.1 app.py - Flask Application Core

Purpose: Initiates the Flask web application server, handles Cross-Origin Resource Sharing (CORS), registers modular Flask Blueprints, and guarantees the SQLite database is initialized with tables on startup.

Key Code Snippets & Functions:

- `database.db.initialize_database()`: Called before starting the server to establish schema.
- `app.register_blueprint(upload_bp)` and `app.register_blueprint(chat_bp)`: Hooks up the routes.
- `CORS(app)`: Allows AJAX requests from frontend templates.

```
from flask import Flask
from flask_cors import CORS
from routes.upload_routes import upload_bp
from routes.chat_routes import chat_bp
from database.db import initialize_database
app = Flask(__name__)
CORS(app)
initialize_database()
```

2.2 config/settings.py - System Configurations

Purpose: Centralizes environment variables using `python-dotenv`. Declares variables for database pathing, upload limits, model models, and keys.

Rotation Enhancements: Added properties `GEMINI_API_KEYS` and `GEMINI_MODELS` to support multi-key and multi-model rotation.

```
class Config:
    SECRET_KEY = os.getenv('SECRET_KEY')
    DATABASE_PATH = os.getenv('DATABASE_PATH', 'data/rag.db')
    PINECONE_API_KEY = os.getenv('PINECONE_API_KEY')
    PINECONE_INDEX_NAME = os.getenv('PINECONE_INDEX_NAME')
    GEMINI_API_KEY = os.getenv('GEMINI_API_KEY')
    GEMINI_API_KEYS = os.getenv('GEMINI_API_KEYS') # Comma-separated list
    GEMINI_MODELS = os.getenv('GEMINI_MODELS') # Fallback models
```

3. Storage & Relational Database Layer

3.1 database/db.py - SQLite Database Orchestrator

Purpose: Establishes localized relational storage. Handles connections and runs transactional queries. Configures foreign key constraints ('PRAGMA foreign_keys = ON;') to trigger cascade deletions.

Tables Structured:

1. **sessions:** Session mapping ('session_id', 'created_at').
2. **documents:** Ingested file mapping ('document_id', 'session_id', 'filename', 'file_path'). References 'sessions' with cascade deletion.
3. **parent_chunks:** Hierarchical text chunks ('parent_id', 'session_id', 'document_id', 'page_number', 'chunk_text'). References 'documents' with cascade deletion.
4. **conversations:** Conversation logging ('id', 'session_id', 'question', 'answer'). Cascade deletes on session purge.

```
@contextmanager
def get_db_connection():
    conn = sqlite3.connect(DATABASE_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute('PRAGMA foreign_keys = ON;')
    try:
        yield conn
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()
```

3.2 services/session_service.py - Session Management

Purpose: Generates secure cryptographically random session tokens prefixed with 'sess_'. Validates session existence and implements session record deletion in SQLite.

```
class SessionService:
    @staticmethod
    def create_session():
        session_id = f'sess_{uuid.uuid4().hex[:8]}'
        execute_insert('INSERT INTO sessions(session_id) VALUES (?)', (session_id,))
        return session_id
```

4. Ingestion Pipeline Services

4.1 services/parser_service.py - PDF Parsing

Purpose: Parses uploaded PDF documents. Iterates page-by-page using `pdfplumber` (falling back to `PyPDF2` if necessary) to extract raw text coordinates and lines.

4.2 services/chunking_service.py - Hierarchical Parent-Child Chunking

Purpose: Splits raw parsed text into structured parent and child chunks. Large parent chunks (~1000 characters) are saved in SQLite to serve as rich generation context. Overlapping smaller child chunks (~200 characters) are embedded and sent to Pinecone to provide fine-grained search indices.

- **create_parent_child_chunks(pages)**: Main loop chunking pages. Generates a map of parent segments and child nodes with page metadata.

4.3 services/embedding_service.py - Text Vectorization

Purpose: Instantiates a HuggingFace `SentenceTransformer` using the BAAI/bge-small-en-v1.5 model. Generates 384-dimensional vector arrays in batches.

4.4 services/pinecone_service.py - Vector Index Operations

Purpose: Interfaces with the Pinecone Vector Database. Performs batch vector uploads, similarity searches restricted by `session\_id`, and purges. Added `delete\_document\_vectors` to query and delete vectors via metadata.

```
@classmethod
def delete_document_vectors(cls, document_id):
    index = cls.get_index()
    index.delete(filter={'document_id': {'$eq': document_id}})
```

4.5 services/ingestion_service.py - Pipeline Orchestrator

Purpose: Orchestrates parsing, chunking, embedding, database inserts, and Pinecone uploads inside a daemon background thread (`threading.Thread`).

State Recovery: Modified status check fallback. If the backend is restarted and the memory registry is empty, it queries SQLite records and returns `completed` status along with page and chunk count metrics.

5. Retrieval and LLM Generation Services

5.1 services/retrieval_service.py - Context Fetching

Purpose: Coordinates retrieval. First queries Pinecone for matching child vector IDs, resolves their associated parent chunks in SQLite, and filters duplicate text fragments.

5.2 services/reranker_service.py - Semantic Reranking

Purpose: Improves retrieval precision. Feeds the retrieved parent chunks and the query into a HuggingFace `CrossEncoder` model (`cross-encoder/ms-marco-MiniLM-L-6-v2`) to re-score and sort context blocks based on their true semantic relevance.

5.3 services/citation_service.py - Sources Extraction

Purpose: Analyzes the final reranked chunks and builds an array of citation references (containing filenames and page numbers) to append as metadata to the AI response.

5.4 services/llm_service.py - Gemini Model with Key/Model Rotation

Purpose: Interfaces with the official `google-genai` client to synthesize context-constrained answers. Implements a robust loop over all configured API keys and models to survive rate-limit exceptions (HTTP 429).

- **initialize_rotation():** Reads `Config.GEMINI\_API\_KEYS` and `Config.GEMINI\_MODELS` and builds lists.
- **generate_answer(...):** Formats RAG context and chat history, then loops through keys/models on error.

```
total_attempts = len(cls._api_keys) * len(cls._models)
for attempt in range(total_attempts):
    api_key = cls._api_keys[cls._current_key_idx]
    model = cls._models[cls._current_model_idx]
    try:
        client = cls.get_client_by_key(api_key)
        response = client.models.generate_content(...)
        # Rotate pointers on success to balance quota usage
        cls.rotate_pointers()
        return response.text
    except Exception as e:
        cls.rotate_pointers() # Advance on failure to attempt failover
```

6. Web API Controllers & Routing

6.1 routes/chat_routes.py - Chat Endpoints

Purpose: Handles chat interactions. Implements blueprints for routing requests:

- **POST** `/api/chat/message`: Resolves retrieval -> reranking -> citations -> LLM rotation generation -> writes history to database.
- **GET** `/api/chat/history/`: Returns historical conversation logs ordered chronologically.
- **DELETE** `/api/chat/session/`: Purges session uploads, Pinecone session vectors, and DB records.

6.2 routes/upload_routes.py - Document Ingestion Controllers

Purpose: Handles files and uploads. Houses routes for session creation and status polling:

- **GET** `/api/upload/documents/`: Lists all ingested documents and their status (used for page refresh recovery).
- **DELETE** `/api/upload/document/`: Removes physical files, purges document vectors, and deletes documents in SQLite.

```
@upload_bp.route('/document/', methods=['DELETE'])
def delete_document(document_id):
    record = fetch_document_metadata(document_id)
    PineconeService.delete_document_vectors(document_id)
    execute_insert('DELETE FROM documents WHERE document_id = ?', (document_id,))
    os.remove(record['file_path'])
    return jsonify({'success': True})
```

7. Presentation & Frontend Layer

7.1 static/js/app.js - UI Controller

Purpose: Front-end orchestrator. Manages page state, calls endpoints, and handles DOM updates.

- **loadSessionHistory()**: Fetches active documents on page load. Resumes status polling and manages chat input states.
- **addDocToList(doc)**: Appends document cards and attaches a trash icon click handler to trigger document deletions.

```
async function deleteDocument(documentId, filename, itemElement) {
    updateStatus(`Deleting ${filename}...`, 'loading');
    const response = await fetch(`/api/upload/document/${documentId}`, { method: 'DELETE' });
    const data = await response.json();
    if (data.success) {
        itemElement.remove();
        checkRemainingDocuments();
    }
}
```

7.2 static/css/style.css & templates/index.html - Dashboard design

Provides a glassmorphic dashboard layout using Outfit typography. Includes styled delete buttons that are hidden by default and fade in smoothly on document item hover.